Smart Expense Tracker System

A robust console-based Java application for tracking personal financial transactions

Made with GAMMA

Project Overview

The Smart Expense Tracker System is a console-based Java application designed to record, track, and summarize personal financial transactions. It offers users a seamless workflow to dynamically input daily expenses and incomes, categorize them, and automatically generate comprehensive financial summaries.

Made with GAMMA

Key Features

Income & Expense Entry

Insert transactions with amounts, categories, and dates

Historical Activity Logging

View comprehensive list of all prior transactions

Dynamic Summarization

Real-time calculations of inflows, outflows, and net balance

Category Search

Filter transactions by targeted category keywords

Made with GAMMA

Income & Expense Entry

01

Input Transaction Details

Enter numerical amounts, named categories, and chronological dates

02

Data Object Creation

System creates new data objects with all transaction details

03

Storage & Organization

Transactions stored with proper categorization and formatting

Made with GAMMA

Historical Activity & Summarization

Activity Logging

Outputs comprehensive list of all prior transactions, clearly labeling them as [INCOME] with positive strings or [EXPENSE] with negative string formats.

Dynamic Summarization

Iterates through transactional memory on demand, returning real-time aggregated calculations of total inflows, outflows, and exact net personal balance.

Made with GAMMA

Object-Oriented Programming Concepts

The system architecture strictly follows robust OOP paradigms throughout construction.

Abstraction

Abstract Transaction.java model encapsulates details (amount, category, date) and enforces abstract display() functionality template.

Inheritance

Income.java and Expense.java extend Transaction.java, reusing super constructor framework and guaranteeing DRY principles.

Polymorphism

Generic display() command called uniformly across arrays, automatically printing correctly formatted elements based on Income or Expense context.

Encapsulation

Internal variables protected using private modifiers, with precise outward access through secure getter parameters ensuring data immutability.

Made with GAMMA

Abstraction & Inheritance

1

Abstract Foundation

Transaction.java model completely encapsulates core details and enforces template mandates

2

Class Extension

Income and Expense classes identically extend Transaction for seamless reuse

3

DRY Principles

Super constructor framework guarantees Don't Repeat Yourself syntax principles

Made with GAMMA

Polymorphism & Encapsulation

Polymorphic Display

Application calls generic display() command uniformly across iterating arrays, automatically printing correctly formatted custom visual elements distinct to Income or Expense subclass context.

Data Protection

Internal variables secured using native private modifiers throughout, with precise outward access through secure internal getter parameters guaranteeing data immutability.

Made with GAMMA

Error Handling & Robustness

Try-Catch Structures

Standard try-catch structures implemented targeting explicit InputMismatchException crashes proactively to prevent runtime failure from flawed text input.

Input Validation

Conditional numerical parameter validations protect program from impossible data vectors, blocking entries where amount <= 0 or empty/null categorical strings.

Made with GAMMA

Technical Stack & Conclusion

Technology Stack

- **Language:** Java
- **Data Structures:** java.util.ArrayList
- **Input Controllers:** java.util.Scanner

Project Success

The application brilliantly structures its foundation mirroring common enterprise paradigms, organizing database layer elements (Transactions), operational service layer (ExpenseManager), and presentation logic (Main) while cleanly tracking financial flows safely and elegantly.

Made with GAMMA